

```

# =====
# INSTALL REQUIRED LIBRARIES
# =====

!pip install sentence-transformers faiss-cpu transformers accelerate sentencepiece gradio -q

# =====
# IMPORT REQUIRED LIBRARIES
# =====

import pandas as pd
import numpy as np
import faiss

from sentence_transformers import SentenceTransformer

from transformers import (
    T5Tokenizer,
    T5ForConditionalGeneration
)

# =====
# LOAD DATASET
# =====

df = pd.read_csv("Cars_DS_Final.csv")

print("\nDataset Loaded Successfully\n")

print(df.head())

# =====
# CREATE TEXT DOCUMENTS
# =====

texts = []

for _, row in df.iterrows():

    text = f"""
    Car Brand: {row.get('Make', '')}
    Car Model: {row.get('Model', '')}
  
```

```

car_model: {row.get('Model', '')}
Variant: {row.get('Variant', '')}
Price: {row.get('Ex-Showroom_Price', '')}
Fuel Type: {row.get('Fuel_Type', '')}
Body Type: {row.get('Body_Type', '')}
Mileage: {row.get('City_Mileage', '')}
Transmission: {row.get('Transmission', '')}
Seating Capacity: {row.get('Seating_Capacity', '')}
"""

texts.append(text)

print(
    "\nText Documents Created :",
    len(texts)
)

# =====
# LOAD EMBEDDING MODEL
# =====

print("\nLoading Embedding Model...\n")

embedding_model = SentenceTransformer(
    "BAAI/bge-large-en-v1.5"
)

# =====
# CREATE EMBEDDINGS
# =====

embeddings = embedding_model.encode(
    texts,
    show_progress_bar=True
)

print("\nEmbeddings Created Successfully")

# =====
# CREATE FAISS INDEX
# =====

embedding_array = np.array(
    embeddings

```

```
).astype("float32")

index = faiss.IndexFlatL2(
    embedding_array.shape[1]
)

index.add(embedding_array)

print(
    "\nFAISS Index Created with",
    index.ntotal,
    "entries"
)

# =====
# RETRIEVAL FUNCTION
# =====

def retrieve(query, k=5):

    query_embedding = embedding_model.encode(
        [query]
    ).astype("float32")

    distances, indices = index.search(
        query_embedding,
        k
    )

    results = []

    for idx in indices[0]:

        results.append(texts[idx])

    return results

# =====
# LOAD FLAN-T5 MODEL
# =====

print("\nLoading FLAN-T5 Model...\n")

tokenizer = T5Tokenizer.from_pretrained(
```

```
"google/flan-t5-base"
)

generator_model = (
    T5ForConditionalGeneration
    .from_pretrained(
        "google/flan-t5-base"
    )
)

print("\nFLAN-T5 Model Loaded Successfully")

# =====
# CHATBOT FUNCTION
# =====

def chatbot(query):

    try:

        # -----
        # RETRIEVE TOP MATCHING CARS
        # -----

        retrieved_docs = retrieve(
            query,
            k=3
        )

        if not retrieved_docs:

            return (
                "Sorry, no matching cars found."
            )

        # -----
        # CREATE CONTEXT
        # -----

        context = ""

        for doc in retrieved_docs:
```

```

context += doc + "\n

# -----
# CREATE PROMPT
# -----

prompt = f"""
User Question:
{query}

Car Details:
{context}

Recommend the best suitable cars.
Mention:
- Price
- Fuel Type
- Mileage
- Body Type
Give short explanation.
"""

# -----
# TOKENIZE INPUT
# -----

inputs = tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True,
    max_length=512
)

# -----
# GENERATE RESPONSE
# -----

outputs = generator_model.generate(
    **inputs,
    max_length=256,
    num_beams=4,
    early_stopping=True
)

```

```

# -----
# DECODE OUTPUT
# -----

final_answer = tokenizer.decode(
    outputs[0],
    skip_special_tokens=True
)

return final_answer

# =====
# ERROR HANDLING
# =====

except Exception as e:

    return (
        f"Error occurred: {str(e)}"
    )

# =====
# MAIN CHATBOT EXECUTION
# =====

print("\n" + "=" * 60)
print("AI CAR RECOMMENDATION CHATBOT")
print("=" * 60)

print("\nType 'exit' to stop the chatbot.\n")

while True:

    query = input(
        "\nAsk about cars: "
    )

    if query.lower() == "exit":

        print(
            "\nChatbot Closed Successfully"
        )

        break

```

```
response = chatbot(query)

print("\n" + "-" * 60)

print("CHATBOT RESPONSE")

print("-" * 60)

print(response)

print("\n" + "=" * 60)
```


aset Loaded Successfully

Unnamed: 0	V1	V2	V3	V4	V5	V6	\
1	NaN	Make	Model	Variant	Ex-Showroom_Price	Displacement	
2	0.0	Tata	Nano Genx	Xt	Rs. 2,92,667	624 cc	
3	1.0	Tata	Nano Genx	Xe	Rs. 2,36,447	624 cc	
4	2.0	Tata	Nano Genx	Emax Xm	Rs. 2,96,661	624 cc	
5	3.0	Tata	Nano Genx	Xta	Rs. 3,34,768	624 cc	

V7	V8	V9	...	\
Cylinders	Valves_Per_Cylinder	Drivetrain	...	
2	2	RWD (Rear Wheel Drive)	...	
2	2	RWD (Rear Wheel Drive)	...	
2	2	RWD (Rear Wheel Drive)	...	
2	2	RWD (Rear Wheel Drive)	...	

V132	V133	V134	\
Leather_Wrapped_Steering	Automatic_Headlamps	Engine_Type	
NaN	NaN	NaN	
NaN	NaN	NaN	
NaN	NaN	NaN	
NaN	NaN	NaN	

V135	V136	V137	V138	\
ASR_/_Traction_Control	Cruise_Control	USB_Ports	Heads-Up_Display	
NaN	NaN	NaN	NaN	
NaN	NaN	NaN	NaN	
NaN	NaN	NaN	NaN	
NaN	NaN	NaN	NaN	

V139	V140	V141
Welcome_Lights	Battery	Electric_Range
NaN	NaN	NaN
NaN	NaN	NaN
NaN	NaN	NaN
NaN	NaN	NaN